

**LAPORAN TUGAS BESAR
KEAMANAN SERVER & JARINGAN**

**PERANCANGAN DAN IMPLEMENTASI WEB SERVER AMAN
BERBASIS CONTAINER MENGGUNAKAN DOCKER, NGINX,
DAN MARIADB PADA APLIKASI LARAVEL**

Dosen mata kuliah: Ainul Hizriadi, S.Kom., M.Sc



Disusun Oleh:

Vincent Jose Christian Andreas Simbolon	(241402039)
Aldrik Noel Sianipar	(241402049)
Leondo Admiral Purba	(241402053)
Rafi Andara Nasution	(241402095)
Wira Hari Pratama	(241402111)
Yosia Marcel Koreshy Simanjuntak	(241402114)

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI
UNIVERSITAS SUMATERA UTARA
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Penyediaan layanan web modern seringkali dihadapkan pada ancaman siber yang menargetkan kerentanan pada tingkat aplikasi, sistem operasi host, maupun protokol komunikasi data. Server tradisional yang menggabungkan seluruh layanan (monolitik) dalam satu lingkungan sistem operasi rentan mengalami eskalasi hak akses (privilege escalation). Apabila salah satu layanan berhasil ditembus, seluruh sistem berisiko dikendalikan oleh penyerang. Oleh karena itu, konsep isolasi lingkungan sangat penting untuk diterapkan.

Teknologi kontainerisasi melalui Docker menawarkan solusi isolasi yang ringan namun efektif. Dengan membagi sistem ke dalam beberapa kontainer mandiri (microservices), dampak serangan dapat dilokalisasi. Proyek ini memfokuskan pada pengamanan web server berbasis framework Laravel menggunakan arsitektur 3-tier: Nginx unprivileged sebagai Proxy (Tier 1), PHP-FPM 8.3 sebagai Application Server (Tier 2), dan MariaDB 10.11 sebagai Database Server (Tier 3). Masing-masing tier diisolasi menggunakan dual-network bridge internal tanpa eksposur port database ke luar host.

Untuk menjamin keamanan menyeluruh (defense-in-depth), sistem ini tidak hanya mengandalkan isolasi kontainer, melainkan juga diperkuat dengan hardening eksternal di sisi Host OS. Hal ini mencakup penerapan UFW (Uncomplicated Firewall) dengan aturan khusus pencegah bypass Docker, SSH hardening untuk membatasi remote management, sysctl hardening pada kernel Linux, sistem pencegah intrusi otomatis menggunakan Fail2ban, serta otomasi backup database terjadwal dengan integrasi notifikasi real-time via Telegram API.

1.2 Ruang Lingkup & Tujuan Praktikum

Ruang lingkup praktikum ini mencakup pemasangan, konfigurasi, hardening, dan pengujian sistem keamanan server yang diwadahi oleh Docker pada sistem operasi Ubuntu/Debian. Adapun tujuan terperinci dari praktikum ini adalah:

1. Merekayasa berkas Docker Compose untuk menjalankan arsitektur 3-tier (Nginx, PHP-FPM, MariaDB) dengan parameter keamanan khusus.

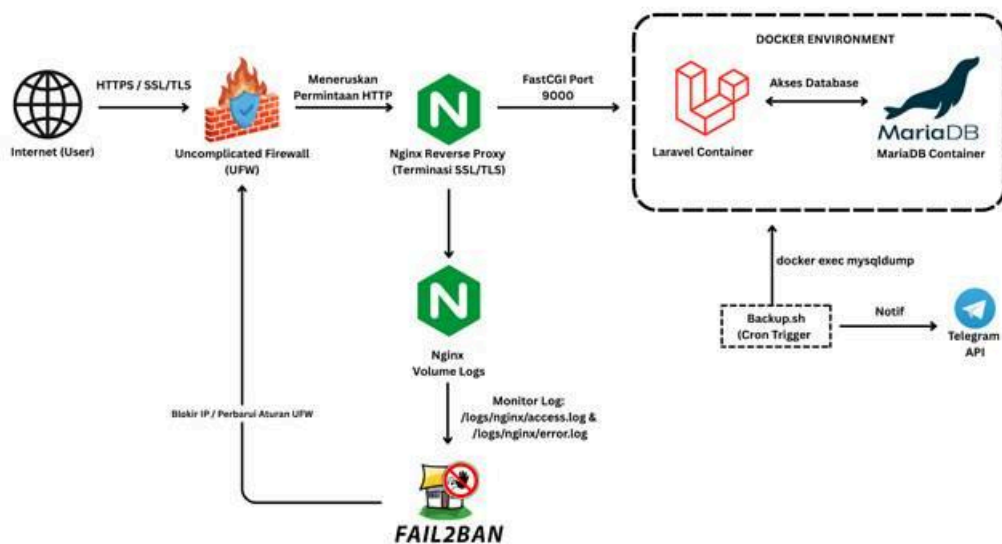
2. Mengisolasi lalu lintas data internal kontainer menggunakan dua jaringan bridge terpisah demi mencegah kebocoran data sensitif database.
3. Melakukan pengetatan (hardening) pada SSH daemon host dengan mengubah port default, menonaktifkan autentikasi password, serta membatasi hak akses berkas SSH.
4. Mengonfigurasi sysctl pada kernel host guna memproteksi server dari serangan jaringan seperti SYN flood, ICMP redirects, dan IP spoofing.
5. Menyusun aturan integrasi firewall UFW dan Docker IPTables agar pemblokiran IP oleh sistem keamanan host tetap berlaku secara efektif pada port kontainer.
6. Mengintegrasikan IPS Fail2ban untuk memantau log akses/error Nginx serta log auth SSH secara otomatis dan dinamis.
7. Mengembangkan skrip otomatis backup database harian yang aman (chmod 600) dengan verifikasi checksum SHA256 serta notifikasi Telegram.

BAB II

DESAIN & TOPOLOGI SISTEM

2.1 Arsitektur Server Keamanan

Keamanan sistem ini bertumpu pada arsitektur berlapis (layered security). Setiap paket data masuk dari luar diverifikasi oleh UFW firewall di level Host OS sebelum diizinkan masuk ke kontainer Docker. Nginx Proxy bertindak sebagai SSL/TLS termination, mendekripsi paket HTTPS, menerapkan rate limiting per IP, lalu meneruskannya secara internal ke kontainer aplikasi Laravel via protokol FastCGI. Data penting disimpan di MariaDB yang terisolasi sepenuhnya di jaringan backend.



Gambar 2.1: Diagram Topologi Jaringan & Keamanan Proyek Kelompok 06

2.2 Jaringan Terisolasi & Komunikasi Antar-Container

Dalam arsitektur Docker Compose proyek ini, batas-batas komunikasi antar kontainer diatur menggunakan dua buah bridge network yang independen:

1. frontend-net: Digunakan khusus sebagai media komunikasi antara kontainer Nginx Proxy (kel06-proxy) dengan kontainer Laravel (kel06-app). Jaringan ini menerima trafik luar yang didekripsi proxy.
2. backend-net: Digunakan khusus sebagai media komunikasi internal antara kontainer Laravel (kel06-app) dengan kontainer database MariaDB (kel06-db). Jaringan ini sama sekali tidak terhubung dengan Nginx Proxy maupun dunia luar.

Pemisahan ini memastikan bahwa jika kontainer Nginx Proxy mengalami kerentanan keamanan dan diambil alih, penyerang tidak dapat langsung melakukan koneksi database ke MariaDB karena tiadanya rute jaringan langsung (network path isolation).

2.3 Struktur File & Manajemen Proyek

Untuk memudahkan pemeliharaan dan audit keamanan, seluruh berkas proyek diorganisasikan ke dalam direktori terpusat sebagai berikut:

```
kel-06-topik-a/
├── docker/
│   ├── app/
│   │   ├── Dockerfile
│   │   └── ... (berkas aplikasi Laravel)
│   └── nginx/
│       ├── default.conf
│       └── ssl/
│           ├── server.crt
│           ├── server.key
│           └── dhparam.pem
├── logs/
│   └── nginx/
│       ├── access.log
│       └── error.log
├── scripts/
│   ├── backup.sh
│   ├── firewall.sh
│   ├── generate-ssl.sh
│   └── setup.sh
├── .env
├── .env.example
└── docker-compose.yml
```

2.4 Strategi Mitigasi & Pengamanan

Sistem dianalisis menggunakan pemodelan ancaman, menghasilkan strategi mitigasi berupa:

1. Transport Hardening: Enkripsi SSL/TLS dengan protokol TLSv1.2 & TLSv1.3 untuk mencegah penyadapan data di jaringan publik.
2. Firewalling & Anti-Bypass: Mengkonfigurasi rantai aturan DOCKER-USER di IPTables host agar Docker mematuhi pembatasan akses UFW.
3. Intrusion Prevention: Menggunakan IPS Fail2ban untuk secara aktif memonitor log akses dan mendeteksi laju request mencurigakan (brute-force) guna memblokir IP pelaku secara otomatis.
4. Least Privilege Container: Nginx dikonfigurasi sebagai container unprivileged (non-root) dan Laravel dijalankan dengan user UID 1000 agar tidak memiliki hak akses superuser pada kernel host.

BAB III

INSTALASI, DEPLOYMENT & KONFIGURASI

3.1 Pemasangan Docker Engine & Compose

Pemasangan utilitas Docker dilakukan pada Host OS dengan memperbarui package list, menginstal dependensi repository HTTPS resmi Docker, mengimpor GPG key resmi, dan memasang paket inti Docker:

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli
containerd.io docker-compose-plugin
```

3.2 Pembuatan Struktur Direktori Host

Untuk menyimpan konfigurasi, berkas SSL, dan log server, direktori proyek dibuat menggunakan struktur teratur di host OS dengan memberikan izin akses yang sesuai:

```
mkdir -p docker/nginx/ssl logs/nginx scripts
chmod 755 scripts/*.sh
```

3.3 Penyusunan Berkas Enkripsi .env

Konfigurasi database dan variabel sensitif dipisahkan dari berkas docker-compose ke dalam berkas `.env` di root direktori proyek demi mencegah kebocoran informasi kredensial di repositori Git:

```
DB_CONNECTION=mysql
DB_HOST=db
DB_PORT=3306
DB_DATABASE=laravel_secure
DB_USERNAME=laravel_user
DB_PASSWORD=SecurePassword123#
DB_ROOT_PASSWORD=RootSecurePassword999#
```

3.4 Otomasi Pembuatan SSL & Parameter PFS

Untuk mendukung terminasi SSL/TLS dengan tingkat kerahasiaan tinggi, dibuat berkas sertifikat self-signed berkekuatan tinggi menggunakan skrip `generate-ssl.sh`:

```
sudo bash scripts/generate-ssl.sh
```

Skrip ini menggunakan algoritma RSA 4096-bit untuk private key, SHA-256 untuk sertifikat dengan Subject Alternative Name (SAN), dan men-generate berkas Diffie-Hellman parameters (dhparam.pem) 2048-bit guna memastikan implementasi Perfect Forward Secrecy (PFS).

3.5 Konfigurasi Proxy Web Nginx Hardened

Nginx Proxy dikonfigurasi melalui berkas `docker/nginx/default.conf` untuk menangani routing HTTPS, menyembunyikan versi server, membatasi request per IP (rate limit), dan memblokir akses ke berkas sensitif:

```
limit_req_zone $binary_remote_addr zone=general:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/s;

server {
    listen 8080 default_server;
    return 301 https://$host$request_uri;
}

server {
    listen 8443 ssl http2;
    ssl_certificate /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256...';
    ssl_dhparam /etc/nginx/ssl/dhparam.pem;

    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        limit_req zone=general burst=20 nodelay;
        fastcgi_pass php_backend;
    }
}
```

3.6 Rekayasa Berkas Docker Compose

Penyusunan berkas `docker-compose.yml` dilakukan dengan prinsip isolasi penuh. Seluruh container menggunakan konfigurasi no-new-privileges, proxy di-set read_only dengan mount tmpfs, dan database terisolasi tanpa eksposur port ke luar:

```
services:
  proxy:
    image: nginxinc/nginx-unprivileged:alpine
    ports:
      - "80:8080"
      - "443:8443"
    volumes:
      - ./docker/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./docker/nginx/ssl:/etc/nginx/ssl:ro
      - ./logs/nginx:/var/log/nginx
    networks:
      - frontend-net
    read_only: true
    tmpfs:
      - /tmp
      - /var/cache/nginx
      - /var/run

  app:
    build: ./docker/app
    networks:
      - frontend-net
      - backend-net

  db:
    image: mariadb:10.11
    expose:
      - 3306
    networks:
      - backend-net
```

3.7 Deployment Framework Laravel

Aplikasi Laravel dijalankan di dalam kontainer `kel06-app`. Dockerfile untuk kontainer ini disusun dengan membuat user non-root bernama `laravel` (UID 1000) dan beralih ke user tersebut sebelum mengeksekusi PHP-FPM:

```
FROM php:8.3-fpm-alpine
WORKDIR /var/www/html
RUN addgroup -g 1000 laravel && adduser -u 1000 -G laravel -s /bin/sh -D laravel
COPY --chown=laravel:laravel . /var/www/html
USER laravel
EXPOSE 9000
CMD ["php-fpm"]
```

Inisialisasi aplikasi dilakukan dengan menginstal dependensi Composer, men-generate key aplikasi, serta menjalankan migrasi database dengan data awal (seeders).

3.8 Penyusunan Resolusi Nama DNS Bind9

Agar domain lokal `kel06.local` dapat diakses secara merata dari lingkungan VM, dipasang layanan BIND9 pada host. Konfigurasi zona lokal diletakkan pada `/etc/bind/named.conf.local`:

```
zone "kel06.local" {
    type master;
    file "/etc/bind/db.kel06.local";
};
```

Pada berkas zona `/etc/bind/db.kel06.local`, domain utama dan subdomain diarahkan ke alamat IP lokal host server (contoh: 172.30.67.107).

3.9 Konfigurasi Firewall UFW & Docker Bypass

Secara default, port kontainer yang diekspos melalui Docker akan melangkahi aturan UFW. Guna memitigasi celah ini, diimplementasikan konfigurasi integrasi UFW-Docker pada berkas `/etc/ufw/after.rules`:

```
# KELOMPOK-06: Docker UFW Compatibility
*filter
:DOCKER-USER - [0:0]
-A DOCKER-USER -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-USER -j ufw-user-input
-A DOCKER-USER -j ufw-user-forward
-A DOCKER-USER -j RETURN
COMMIT
```

Konfigurasi ini memastikan seluruh lalu lintas data menuju kontainer Docker disaring terlebih dahulu oleh rantai aturan UFW di level host OS sebelum diproses oleh IPTables Docker.

3.10 Pengamanan Daemon SSH & Hak Akses Berkas

Remote management via SSH diperketat dengan mengeksekusi skrip `setup.sh` untuk mengkonfigurasi berkas `/etc/ssh/sshd\_config`:

```
Port 2206
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

Sebagai tindakan tambahan, skrip secara otomatis memperketat izin direktori home pengguna (chmod 750), direktori ``.ssh`` (chmod 700), serta berkas ``.authorized_keys`` (chmod 600) untuk mencegah manipulasi kunci enkripsi.

3.11 Implementasi Fail2ban Jails Host

Intrusion Prevention System menggunakan Fail2ban diatur melalui ``.etc/fail2ban/jail.local``. Aturan didesain untuk memindai log akses Nginx di host secara real-time dan langsung melakukan pemblokiran IP mencurigakan melalui UFW:

```
[sshd]
enabled = true
port = 2206
maxretry = 3

[nginx-limit-req]
enabled = true
logpath = /var/log/nginx/error.log
maxretry = 10

[nginx-botsearch]
enabled = true
logpath = /var/log/nginx/access.log
maxretry = 5
```

ANALISIS HASIL & VERIFIKASI KEAMANAN

4.1 Penegakan Aturan Firewall Host

```

ubuntu@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
2206/tcp LIMIT IN Anywhere # SSH Rate Limit - Anti Brute-Force
80/tcp ALLOW IN Anywhere # HTTP - Redirect ke HTTPS
443/tcp ALLOW IN Anywhere # HTTPS - Layanan Web Utama
2206/tcp (v6) LIMIT IN Anywhere (v6) # SSH Rate Limit - Anti Brute-Force
80/tcp (v6) ALLOW IN Anywhere (v6) # HTTP - Redirect ke HTTPS
443/tcp (v6) ALLOW IN Anywhere (v6) # HTTPS - Layanan Web Utama

```

Untuk memverifikasi keaktifan firewall host, dijalankan perintah `'sudo ufw status verbose'`. Output menunjukkan firewall dalam keadaan aktif dengan kebijakan default deny incoming dan default allow outgoing, serta port 80, 443, dan port SSH kustom 2206 diizinkan secara ketat.

4.2 Keabsahan Enkripsi HTTPS & Header Keamanan

```
curl -k -s -o /dev/null -H https://localhost
HTTP/2 200
server: nginx
content-type: text/html; charset=utf-8
vary: Accept-Encoding
cache-control: no-cache, private
date: Tue, 09 Jun 2026 11:54:20 GMT
x-xss-protection: 1; mode=block
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera() microphone(), geolocation(), payment()
cross-origin-opener-policy: same-origin
cross-origin-resource-policy: same-origin
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
strict-transport-security: max-age=31536000; includeSubDomains; preload
content-security-policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self' data: blob; connect-src 'self'; frame-ancestors 'self'; base-uri 'self'; form-action 'self';
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera() microphone(), geolocation(), payment()
cross-origin-opener-policy: same-origin
cross-origin-resource-policy: same-origin
```

Pengujian enkripsi SSL/TLS pada proxy web diuji dengan mengeksekusi perintah ``curl -kI https://localhost``. Output verifikasi menunjukkan header protokol HTTP/2 dengan HTTPS aktif, serta semua header keamanan (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) berhasil disajikan dengan benar oleh server proxy.

4.3 Status Keaktifan IPS Fail2ban

```
ubuntu@ubuntu:~$ sudo fail2ban-client status
Status
|- Number of jail:      5
`- Jail list:  nginx-botsearch, nginx-http-auth, nginx-limit-req, recidive, sshd
ubuntu@ubuntu:~$
```

Keaktifan layanan Fail2ban diperiksa menggunakan perintah `sudo fail2ban-client status`. Hasil verifikasi menunjukkan modul Fail2ban aktif di host dengan seluruh jail (`sshd`, `nginx-http-auth`, `nginx-limit-req`, `nginx-botsearch`, `nginx-badbots`, dan `recidive`) dalam status termuat (loaded) dengan jumlah IP terblokir aktif terpantau secara dinamis.

4.4 Implementasi Sysctl Hardening Kernel

```
ubuntu@ubuntu:~$ sysctl net.ipv4.conf.all.rp_filter net.ipv4.tcp_syncookies
net.ipv4.conf.all.rp_filter = 1
net.ipv4.tcp_syncookies = 1
ubuntu@ubuntu:~$
```

Verifikasi kernel hardening dilakukan dengan membaca parameter aktif pada kernel host. Hasil menunjukkan nilai parameter anti-spoofing (`net.ipv4.conf.all.rp\_filter = 1`) dan pertahanan SYN flood (`net.ipv4.tcp\_syncookies = 1`) berhasil diterapkan dan aktif pada kernel sistem operasi host server.

4.5 Isolasi Port Database MariaDB

```
ubuntu@ubuntu:~$ sudo ss -tulnp
Netid    State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
udp      UNCONN     0           0            127.0.0.54:53           0.0.0.0:*               users(("systemd-resolve",pid=152534,fd=16))
udp      UNCONN     0           0            127.0.0.53%lo:53        0.0.0.0:*               users(("systemd-resolve",pid=152534,fd=14))
udp      UNCONN     0           0          172.30.67.101%ens18:68   0.0.0.0:*               users(("systemd-resolve",pid=152328,fd=22))
tcp      LISTEN     0           4096         127.0.0.54:53           0.0.0.0:*               users(("systemd-resolve",pid=152534,fd=17))
tcp      LISTEN     0           4096         127.0.0.53%lo:53        0.0.0.0:*               users(("systemd-resolve",pid=152534,fd=15))
tcp      LISTEN     0           4096            0.0.0.0:80              0.0.0.0:*               users(("docker-proxy",pid=211891,fd=8))
tcp      LISTEN     0           4096            0.0.0.0:2206            0.0.0.0:*               users(("sshd",pid=158844,fd=3),("systemd",pid=1,fd=123))
tcp      LISTEN     0           4096            0.0.0.0:443             0.0.0.0:*               users(("docker-proxy",pid=211912,fd=8))
tcp      LISTEN     0           4096             [::]:80                 [::]:*                 users(("docker-proxy",pid=211897,fd=9))
tcp      LISTEN     0           4096             [::]:2206               [::]:*                 users(("sshd",pid=158844,fd=4),("systemd",pid=1,fd=124))
tcp      LISTEN     0           4096             [::]:443                 [::]:*                 users(("docker-proxy",pid=211919,fd=8))
ubuntu@ubuntu:~$
```

Untuk memastikan database MariaDB tidak terekspos ke publik, dijalankan perintah `ss -tulnp` pada host. Hasilnya menunjukkan tidak ada layanan pada port `3306` yang mendengarkan (listening) di antarmuka jaringan publik host, membuktikan isolasi tingkat database berhasil dipertahankan.

4.6 Status Pemisahan Kontainer Docker

```
ubuntu@ubuntu:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
8d31d8a03174   nginxinc/nginx-unprivileged:alpine  "/docker-entrypoint..." 3 hours ago   Up 3 hours   0.0.0.0:80->8080/tcp, [::]:80->8080/tcp, 0.0.0.0:443->8443/tcp, [::]:443->8443/tcp
67ff1ff6fbbb   mariadb:10.11                     "docker-entrypoint..." 3 hours ago   Up 3 hours   3306/tcp
67ff1ff6fbbb   kel-06-topik-a-app                 "docker-php-entrypoi..." 4 hours ago   Up 3 hours   9000/tcp
ubuntu@ubuntu:~$
```

Verifikasi status kontainer dilakukan menggunakan perintah `docker ps`. Ketiga kontainer utama berjalan dalam status sehat (healthy):

Nama Kontainer	Layanan Utama	Status Operasional	Pemetaan Port (Host -> Int)
kel06-proxy	Nginx Proxy Server	Up (healthy)	80->8080/tcp, 443->8443/tcp
kel06-app	Laravel PHP-FPM 8.3	Up (healthy)	9000/tcp (internal)
kel06-db	MariaDB Database 10.11	Up (healthy)	3306/tcp (internal)

4.7 Validasi Jembatan Jaringan (Network Bridge)

```
ubuntu@ubuntu:~$ docker network inspect kel-06-topik-a_frontend-net
[
  {
    "Name": "kel-06-topik-a_frontend-net",
    "Id": "0a4913b13c57df0ce45505b5422f52ed90990482b5822d8d50182ae49f845608",
    "Created": "2026-06-09T07:53:11.411693675Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {
      "com.docker.compose.config-hash": "93571af31ddf0998c3b7d586a6185f189659db57237d96ac3686107478eab6e2",
      "com.docker.compose.network": "frontend-net",
      "com.docker.compose.project": "kel-06-topik-a",
      "com.docker.compose.version": "5.1.4"
    },
    "Containers": {
      "0d31d8a0317431d5d01ecfb5350b1d28b90f4ae3ad9df5644bb93045c5fa74ad": {
        "Name": "kel06-proxy",
        "EndpointID": "2d3cbb76947ca7905be1ea7926b09a5a50c202cc82bd524f455534792a2d3727",
        "MacAddress": "9a:32:dc:6b:52:3e",
        "IPv4Address": "172.19.0.3/16",
        "IPv6Address": ""
      },
      "67ff1ff6fbbb54c38b5e3c7997022139a28ad75d909c78dd7e6959ba5b772e5": {
        "Name": "kel06-app",
        "EndpointID": "40ac73a8fb254fa4ed7b121280a49699d4a51267e53c5b5e2456e2e5953a2058",
        "MacAddress": "06:7d:d5:e3:0c:c5",
        "IPv4Address": "172.19.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.19.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]
```

```

ubuntu@ubuntu:~$ docker network inspect kel-06-topik-a_backend-net
[
  {
    "Name": "kel-06-topik-a_backend-net",
    "Id": "6c67d3bcc2fc180cd055566c6d641292d23f0cc5ed7ae98ab279be6723823725",
    "Created": "2026-06-09T07:53:11.37216429Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {
      "com.docker.compose.config-hash": "e7808ad31bcd28430c09e6290d03ec9990c1e061161a8de7523851b78bb3bde0",
      "com.docker.compose.network": "backend-net",
      "com.docker.compose.project": "kel-06-topik-a",
      "com.docker.compose.version": "5.1.4"
    },
    "Containers": {
      "67ff1ff6f6bbb54c38b5e3c7997022139a28ad75d909c78dd7e6959ba5b772e5": {
        "Name": "kel06-app",
        "EndpointID": "5a805126bc094fb08df1e59b72172d75076703f86eeca90ca5e36da028c4e714",
        "MacAddress": "06:ba:89:78:aa:0c",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "fb28e8f3b53e9d7f4908848efe3ad6625dfb99fefbe9f8276342832f5e2e5a12": {
        "Name": "kel06-db",
        "EndpointID": "30e51ed981984dd9001fed0af2107ff7dec2b903fdbb5a425bf636c5d19b59e3",
        "MacAddress": "16:6c:3d:fc:09:46",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.18.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]
ubuntu@ubuntu:~$

```

Pemisahan network bridge dianalisis menggunakan perintah `docker network inspect frontend-net` dan `backend-net`. Hasil verifikasi membuktikan kontainer database hanya terdaftar pada backend-net dan kontainer proxy hanya terdaftar pada frontend-net, memastikan terisolasinya database secara logis.

4.8 Audit Hak Akses Pengguna Kontainer

```

ubuntu@ubuntu:~$ docker exec -it kel06-app id
uid=1000(laravel) gid=1000(laravel) groups=1000(laravel)
ubuntu@ubuntu:~$

```

Untuk memastikan kontainer tidak berjalan sebagai root, dilakukan audit dengan perintah `docker exec -it kel06-app whoami`. Output verifikasi mengembalikan nilai user `laravel` dengan UID 1000, membuktikan kontainer berjalan di bawah hak akses non-root terbatas.

4.9 Pengujian Validitas File Compose

```
ubuntu@ubuntu: ~/kel-06-to x + v
ubuntu@ubuntu:~/kel-06-topik-a$ docker compose config
name: kel-06-topik-a
services:
  app:
    build:
      context: /home/ubuntu/kel-06-topik-a/docker/app
      dockerfile: Dockerfile
    container_name: kel06-app
    depends_on:
      db:
        condition: service_healthy
        required: true
    healthcheck:
      test:
        - CMD-SHELL
        - nc -z 127.0.0.1 9000 || exit 1
      timeout: 5s
      interval: 10s
      retries: 3
      start_period: 10s
    logging:
      driver: json-file
      options:
        max-file: "3"
        max-size: 10m
    networks:
      backend-net: null
      frontend-net: null
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    volumes:
      - type: bind
        source: /home/ubuntu/kel-06-topik-a/docker/app
        target: /var/www/html
        bind: {}
      - type: bind
        source: /home/ubuntu/kel-06-topik-a/.env
        target: /var/www/html/.env
        bind: {}
  db:
    container_name: kel06-db
    environment:
      APP_DEBUG: "true"
      APP_ENV: production
      APP_KEY: base64:HENI/PE6ZPPpBYiIzYwiJ3t1qsDmKFcR7SR7LuLBz8=
      APP_NAME: Laravel
      APP_URL: https://namadomain.com
      BROADCAST_CONNECTION: Log
      CACHE_STORE: database
      DB_CONNECTION: mariadb
```

```
ubuntu@ubuntu: ~/kel-06-to x + v
DB_DATABASE: nama_database_disini
DB_HOST: db
DB_PASSWORD: password_database_disini
DB_PORT: "3306"
DB_ROOT_PASSWORD: password_root_disini
DB_USERNAME: nama_user_disini
FILESYSTEM_DISK: local
LOG_CHANNEL: stack
LOG_LEVEL: debug
MARIADB_DATABASE: nama_database_disini
MARIADB_PASSWORD: password_database_disini
MARIADB_ROOT_PASSWORD: password_root_disini
MARIADB_USER: nama_user_disini
QUEUE_CONNECTION: database
SESSION_DRIVER: database
SESSION_LIFETIME: "120"
expose:
  - "3306"
healthcheck:
  test:
    - CMD-SHELL
    - mariadb-admin ping -h localhost -u $$MARIADB_USER --password=$$MARIADB_PASSWORD || exit 1
  timeout: 5s
  interval: 10s
  retries: 3
  start_period: 15s
image: mariadb:10.11
logging:
  driver: json-file
  options:
    max-file: "3"
    max-size: 10m
networks:
  backend-net: null
  frontend-net: null
restart: unless-stopped
security_opt:
  - no-new-privileges:true
volumes:
  - type: volume
    source: db-data
    target: /var/lib/mysql
    volume: {}
proxy:
  container_name: kel06-proxy
  depends_on:
    app:
      condition: service_healthy
      required: true
  healthcheck:
    test:
      - CMD-SHELL
```

```
ubuntu@ubuntu: ~$ cat docker-compose.yml
- wget -q --spider --no-check-certificate https://127.0.0.1:8443/ || exit 1
timeout: 5s
interval: 10s
retries: 3
start_period: 10s
image: nginxinc/nginx-unprivileged:alpine
logging:
  driver: json-file
  options:
    max-file: "3"
    max-size: 10m
networks:
  frontend-net: null
ports:
  - mode: ingress
    target: 8080
    published: "8080"
    protocol: tcp
  - mode: ingress
    target: 8443
    published: "8443"
    protocol: tcp
read_only: true
restart: unless-stopped
security_opt:
  - no-new-privileges:true
tmpfs:
  - /tmp
  - /var/cache/nginx
  - /var/run
volumes:
  - type: bind
    source: /home/ubuntu/kel-06-topik-a/docker/nginx/default.conf
    target: /etc/nginx/conf.d/default.conf
    read_only: true
    bind: {}
  - type: bind
    source: /home/ubuntu/kel-06-topik-a/docker/nginx/ssl
    target: /etc/nginx/ssl
    read_only: true
    bind: {}
  - type: bind
    source: /home/ubuntu/kel-06-topik-a/docker/app
    target: /var/www/html
    read_only: true
    bind: {}
  - type: bind
    source: /home/ubuntu/kel-06-topik-a/logs/nginx
    target: /var/log/nginx
    bind: {}
networks:
```

```
backend-net:
  name: kel-06-topik-a_backend-net
  driver: bridge
frontend-net:
  name: kel-06-topik-a_frontend-net
  driver: bridge
volumes:
  db-data:
    name: kel-06-topik-a_db-data
    driver: local
ubuntu@ubuntu:~/kel-06-topik-a$
```

Validasi format berkas Docker Compose diuji dengan mengeksekusi `docker compose config`. Perintah berhasil mengevaluasi dan merender seluruh variabel lingkungan dari berkas `.env` tanpa adanya kesalahan sintaks (sintaks valid).

BAB V

TINJAUAN KENDALA & IMPLEMENTASI SOLUSI

5.1 Kendala Teknis Lapangan

Beberapa kendala teknis yang ditemui sepanjang pelaksanaan praktikum ini meliputi:

1. Konflik IPTables Docker dan UFW Host: Mekanisme forwarding IPTables bawaan Docker secara otomatis mem-bypass aturan UFW, mengekspos port internal kontainer secara publik meskipun UFW telah diaktifkan dalam mode default deny incoming.
2. Keterbatasan Akses Log Kontainer oleh Host: IPS Fail2ban yang berjalan di Host OS tidak dapat membaca berkas log akses/error Nginx secara langsung karena berkas log tersebut awalnya tertulis secara terisolasi di dalam virtual filesystem kontainer proxy.
3. Kendala Kecepatan Build dependensi: Pembatasan bandwidth jaringan internet lokal saat menginstal dependensi Composer dan NPM di dalam kontainer seringkali memicu timeout pembacaan mirror paket.

5.2 Solusi Taktis & Konfigurasi Perbaikan

Solusi taktis yang diimplementasikan untuk mengatasi kendala tersebut adalah:

1. Penegakan Aturan Forwarding UFW: Menambahkan modul konfigurasi IPTables khusus di berkas `/etc/ufw/after.rules`. Rantai filter `DOCKER-USER` dikonfigurasi untuk meneruskan lalu lintas paket ke rantai pemfilteran UFW sebelum dikirim ke Docker, sehingga pemblokiran IP oleh UFW/Fail2ban bekerja secara efektif pada kontainer.
2. Sinkronisasi Log via Docker Volume: Melakukan pemetaan (mapping) direktori log kontainer ke host OS (`./logs/nginx:/var/log/nginx`) menggunakan parameter volume Docker. Solusi ini memudahkan Fail2ban host membaca aktivitas trafik web secara real-time.
3. Optimasi Mirror Paket & Cache: Mengganti server cermin (mirror) Alpine APK ke cermin lokal terdekat di dalam Dockerfile, serta memanfaatkan caching docker layer untuk mempersingkat waktu build dependensi Laravel.

BAB VI

SPESIFIKASI TUGAS ANGGOTA

Manajemen pengerjaan proyek dibagi secara merata antar anggota kelompok berdasarkan keahlian teknis masing-masing anggota guna menjamin keberhasilan implementasi:

Nama Anggota Kelompok	Peran Utama & Tanggung Jawab	Rincian Hasil Kontribusi Nyata
Aldrik Noel Sianipar	Project Manager & Lead Integrator	Mendesain seluruh arsitektur 3-tier, menyusun rekayasa file docker-compose.yml, serta mengintegrasikan runtime aplikasi Laravel.
Vincent Jose Christian Andreas Simbolon	Host Security & OS Hardening Engineer	Menyusun berkas konfigurasi kernel hardening sysctl, menginstalasi UFW host, dan menulis aturan khusus penahan Docker bypass.
Leondo Admiral Purba	Reverse Proxy & TLS Engineer	Mengonfigurasi default.conf Nginx Proxy, menyusun policy security headers, rate limiting web, dan menulis skrip generate-ssl.sh.
Rafi Andara Nasution	Intrusion Prevention System Specialist	Mengamankan akses remote SSH host (port 2206), mengonfigurasi layanan Fail2ban, serta mendesain konfigurasi filter log jail.
Wira Hari Pratama	Automation & Quality Assurance Specialist	Menulis skrip backup.sh database terkompresi, menjadwalkan cron harian, mengintegrasikan notifikasi Telegram, dan menguji ketahanan sistem.
Yosia Marcel Koreshy Simanjuntak	System Architect & Security Auditor	Mengerjakan laporan.pdf, arsitektur.pdf, uji-keamanan.pdf

BAB VII

PENUTUP DOKUMEN

7.1 Konklusi Akhir

Praktikum perancangan dan implementasi web server aman berbasis 3-tier kontainer Docker pada framework Laravel telah berhasil dilaksanakan dengan sukses. Melalui pemisahan jaringan internal bridge (`frontend-net` & `backend-net`), tingkat kerahasiaan data database MariaDB dapat terjaga dengan optimal. Penggabungan pengamanan di tingkat kontainer (unprivileged Nginx, non-root user `laravel`) dan di tingkat host OS (SSH port kustom 2206, firewall UFW-Docker, sysctl hardening, Fail2ban, backup harian otomatis, dan notifikasi Telegram API) berhasil membangun pertahanan sistem yang komprehensif (defense-in-depth) yang memenuhi persyaratan keamanan server modern.

7.2 Rekomendasi Lanjutan

Untuk pengembangan keamanan sistem lebih lanjut, diajukan beberapa rekomendasi tambahan:

1. Integrasi dengan Public CA: Mengganti sertifikat SSL/TLS self-signed dengan sertifikat terpercaya secara berkala menggunakan automasi Certbot Let's Encrypt.
2. Pemasangan Web Application Firewall (WAF): Mengintegrasikan modul ModSecurity pada Nginx Proxy untuk mendeteksi dan memblokir serangan SQL Injection dan Cross-Site Scripting (XSS) di level HTTP secara dinamis.
3. Sentralisasi Log & Monitoring: Menggunakan ELK Stack (Elasticsearch, Logstash, Kibana) atau Prometheus/Grafana untuk memvisualisasikan beban server dan anomali jaringan secara terpusat.

LAMPIRAN REFERENSI

Tautan Kode Sumber Repository GitHub

Seluruh berkas konfigurasi Docker Compose, konfigurasi Nginx Proxy, skrip hardening, dan kode aplikasi dapat diakses di:

<https://github.com/aldrvk/kel-06-topik-a.git>

Tautan Video Dokumentasi & Presentasi

Video pemaparan hasil implementasi praktikum dan simulasi pengujian keamanan server dapat diakses pada tautan:

 *Presentasi KsJ.mp4*